

```

r, d = map(int, input().split())
a = [[int(x) for x in input().split()] for y in range(n)]
import sys
sys.stdin = open('input.txt', 'r')
sys.stdout = open('output.txt', 'w')

```

Algebra Formulas

If a and r are real numbers and $r \neq 0$, then

$$\sum_{j=0}^n ar^j = \begin{cases} \frac{ar^{n+1} - a}{r - 1} & \text{if } r \neq 1 \\ (n+1)a & \text{if } r = 1. \end{cases}$$

$\sum_{k=1}^n k$	$\frac{n(n+1)}{2}$
$\sum_{k=1}^n k^2$	$\frac{n(n+1)(2n+1)}{6}$
$\sum_{k=1}^n k^3$	$\frac{n^2(n+1)^2}{4}$
$\sum_{k=0}^{\infty} x^k, x < 1$	$\frac{1}{1-x}$
$\sum_{k=1}^{\infty} kx^{k-1}, x < 1$	$\frac{1}{(1-x)^2}$

- **LCM * GCD** = the multiple of the 2 numbers
- **LCM** = multiply the numbers which has the greatest power from the 2 numbers even if they're not common
- **GCD** = multiply the numbers which has the lowest power from the 2 numbers even if they're not common

- **Arithmetic sequence** : $\frac{n}{2} (a + b)$

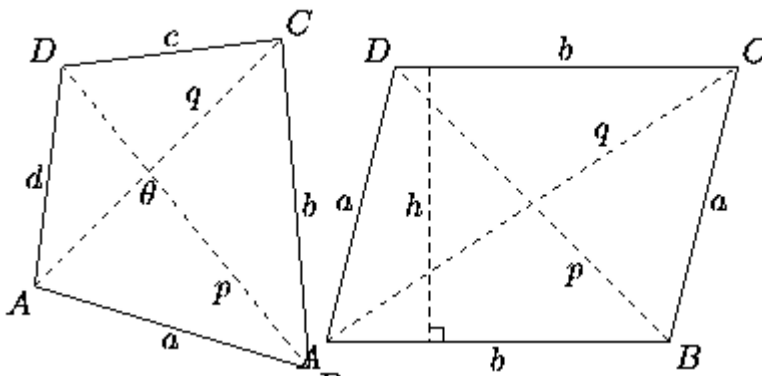
- **Geometric sequence** : $\frac{L * R - a}{r - 1} = a * \frac{r^n - 1}{r - 1}$

- **Sum of odd numbers** : $\left(\frac{n + n \% 2}{2}\right)^2$

- Sum of even numbers: $\frac{(\frac{n}{2} * (2 + n - n \% 2))}{2}$
- Quadratic Formula : $\frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$

Geometry Formulas

- Number of diagonals = $\frac{n(n-3)}{2}$



- Area = $\frac{1}{2}PQ \sin(\theta) = \frac{1}{4}(b^2 + d^2 - a^2 + c^2)\tan(\theta) =$
 $\frac{1}{4}\sqrt{4P^2Q^2 - (b^2 + d^2 - a^2 - c^2)^2} =$
 $\sqrt{(s-a)(s-b)(s-c)(s-d) - abcd\cos\frac{1}{2}(A+C)}$

- Area of sector = $\frac{1}{2}r^2\theta_{rad}$

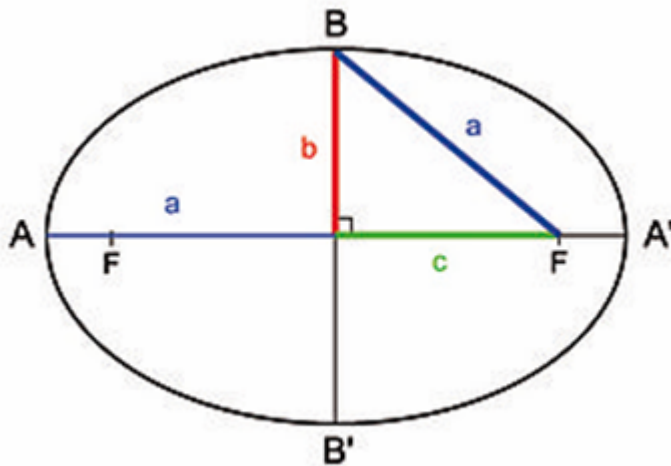
- Area of triangle = $\sqrt{p(p-a)(p-b)(p-c)}$, where a, b, c are the lengths of the three sides and $p = \frac{a+b+c}{2} =$

$$\frac{absin(C)}{2} = \left| \frac{Ax(By-Cy) + Bx(Cy-Ay) + Cx(Ay-By)}{2} \right| = \sqrt{\frac{3}{4}}S^2,$$

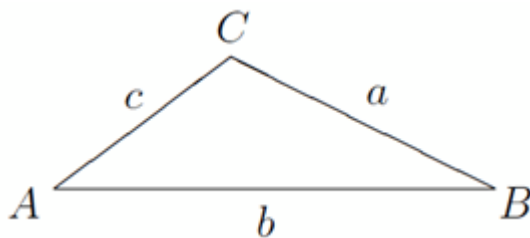
where s is any side of an equilateral triangle =

$$\frac{A \times B + B \times C + C \times A}{2}, \text{ where A, B, C are the three vectors}$$

- Cross product equals area of parallelogram

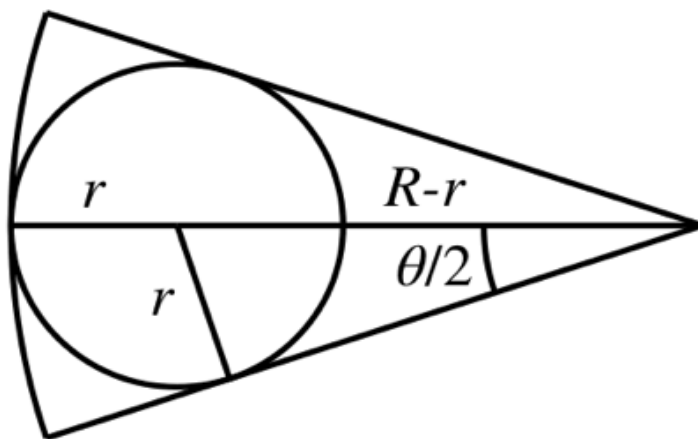


- Area of an ellipse: $ab\pi$
- Area of trapezium: $\frac{a+b}{2}h$
- The volume of cone: $\frac{h}{3}\pi r^2$
- The volume of Pyramid: $\frac{abh}{3}$
- Lengths of a right-angled triangle given non-hypot side:
 $\frac{n^2}{4}, \frac{n^2}{4} + 1, n, n$ is even, $\frac{n^2}{2}, \frac{n^2}{2} + 1, n, n$ is odd
- Chord length: $2r\sin(\frac{\theta}{2})$



- $c^2 = a^2 + b^2 - 2ab \cos(C)$

- $\theta^\circ = \frac{\theta^{rad} * 180}{\Pi}$



says

$$r = (R - r) \sin\left(\frac{\theta}{2}\right)$$

Therefore, we can solve for r :

$$r = R \frac{\sin\left(\frac{\theta}{2}\right)}{1 + \sin\left(\frac{\theta}{2}\right)}$$

To find θ from R and r , we can use

$$\sin\left(\frac{\theta}{2}\right) = \frac{r}{R - r}$$

-

Combinatorics Formulas

- $(1 + t)^n = \sum_{k=0}^n \binom{n}{k} t^k, t = 1 \Rightarrow 2^n$

- $(x + y)^n = \sum_{k=0}^n \binom{n}{k} x^{n-k} y^k = \sum_{k=0}^n x^k y^{n-k}$

- $\left(1 + \frac{1}{n}\right)^n = 1 + \binom{n}{1} \frac{1}{n} + \binom{n}{2} \frac{1}{n^2} + \binom{n}{3} \frac{1}{n^3} + \dots + \binom{n}{n} \frac{1}{n^n}.$

- $\binom{n}{k} \frac{1}{n^k} = \frac{1}{k!} \cdot \frac{n(n-1)(n-2) \cdots (n-k+1)}{n^k}$
- Permutation in circle = nPr / r
- Number of ways to distribute n distinct objects around r identical boxes such that each box has at least one object are denoted by $s(n, r)$:
- $s(n, r) = r * s(n-1, r) + s(n-1, r-1)$
 1. $r > n \implies S(n, r) = 0.$
 2. $r = n \implies S(n, n) = 1.$
 3. $r = n-1 \implies S(n, n-1) = \binom{n}{2}.$
 4. $r = n-2 \implies S(n, n-2) = \binom{n}{3} + 3\binom{n}{4}.$
 5. $r = 1 \implies S(n, 1) = 1.$
 6. $r = 2 \implies S(n, 2) = 2^{n-1} - 1.$
- The number of ways to reach cell (n, m) from $(1, 1) = (n+m-1)C_{m-1}$
- The number of r -combinations of $x = \{1, 2, 3, \dots, n\}$ with no consecutive integers = $(n-r+1)C_r$
- The number of ways to distribute n objects into m boxes, such that box i has n_i objects:

$$\binom{n}{n_1, n_2, \dots, n_m} = \frac{n!}{n_1! n_2! \cdots n_m!}.$$

$$\binom{n}{r} = \binom{n}{n-r} \quad (2.1.1)$$

$$\binom{n}{r} = \frac{n}{r} \binom{n-1}{r-1} \quad \text{provided that } r \geq 1 \quad (2.1.2)$$

$$\binom{n}{r} = \frac{n-r+1}{r} \binom{n}{r-1} \quad \text{provided that } r \geq 1 \quad (2.1.3)$$

$$\binom{n}{r} = \binom{n-1}{r-1} + \binom{n-1}{r} \quad (2.1.4)$$

$$\binom{n}{m} \binom{m}{r} = \binom{n}{r} \binom{n-r}{m-r} \quad (2.1.5)$$

$$\binom{n}{n_1, n_2, \dots, n_m} = \frac{n!}{n_1! n_2! \dots n_m!}.$$

Let p be a prime. Then

- (i) $\binom{n}{p} \equiv \lfloor \frac{n}{p} \rfloor \pmod{p}$ for every $n \in \mathbb{N}$,
- (ii) $\binom{p}{r} \equiv 0 \pmod{p}$ for every r such that $1 \leq r \leq p-1$,
- (iii) $\binom{p+1}{r} \equiv 0 \pmod{p}$ for every r such that $2 \leq r \leq p-1$,
- (iv) $\binom{p-1}{r} \equiv (-1)^r \pmod{p}$ for every r such that $0 \leq r \leq p-1$,
- (v) $\binom{p-2}{r} \equiv (-1)^r (r+1) \pmod{p}$ for every r such that $0 \leq r \leq p-2$,
- (vi) $\binom{p-3}{r} \equiv (-1)^r \binom{r+2}{2} \pmod{p}$ for every r such that $0 \leq r \leq p-3$.

communication) that an integer $n > 2$ is prime iff

$$n \mid \binom{n}{6k \pm 1},$$

for all k with $1 \leq k \leq \lfloor \frac{\sqrt{n}}{6} \rfloor$, where $\lfloor x \rfloor$ denotes the greatest integer not exceeding the real number x .

n is a prime iff $n \mid \binom{n}{r}$ for all $r = 1, 2, \dots, n-1$.

Reflection Principle (RP). The number of shortest routes from P to Q that meet the line L is equal to the number of shortest routes from P' to Q .

$$(i) \quad \binom{r}{r} + \binom{r+1}{r} + \cdots + \binom{n}{r} = \binom{n+1}{r+1} \quad (2.5.1)$$

for all $r, n \in \mathbb{N}$ with $n \geq r$;

$$(ii) \quad \binom{r}{0} + \binom{r+1}{1} + \cdots + \binom{r+k}{k} = \binom{r+k+1}{k} \quad (2.5.2)$$

for all $r, k \in \mathbb{N}$.

$$\begin{aligned} \sum_{i=0}^r \binom{m}{i} \binom{n}{r-i} &= \binom{m}{0} \binom{n}{r} + \binom{m}{1} \binom{n}{r-1} + \cdots + \binom{m}{r} \binom{n}{0} \\ &= \binom{m+n}{r}. \end{aligned} \quad (2.3.5)$$

$$\sum_{r=1}^n r^2 \binom{n}{r} = n(n+1)2^{n-2},$$

$$\sum_{r=1}^n r^3 \binom{n}{r} = n^2(n+3)2^{n-3}$$

$$\sum_{r=1}^n r \binom{n}{r} = \binom{n}{1} + 2\binom{n}{2} + 3\binom{n}{3} + \cdots + n\binom{n}{n} = n \cdot 2^{n-1}. \quad (2.3.4)$$

$$(i) \sum_{r=0}^n (-1)^r \binom{n}{r} = \binom{n}{0} - \binom{n}{1} + \binom{n}{2} - \cdots + (-1)^n \binom{n}{n} = 0, \quad (2.3.2)$$

$$(ii) \binom{n}{0} + \binom{n}{2} + \cdots + \binom{n}{2k} + \cdots = \binom{n}{1} + \binom{n}{3} + \cdots + \binom{n}{2k+1} + \cdots = 2^{n-1}. \quad (2.3.3)$$

- The number of derangements, where r is the number of elements selected and k is the number of fixed points:

$$D(n, r, k) = \frac{\binom{r}{k}}{(n-r)!} \sum_{i=0}^{r-k} (-1)^i \binom{r-k}{i} (n-k-i)! .$$

Pascals Triangle

																1																																																			
															1			1																																																	
														1			2			1																																															
													1			3			3			1																																													
												1			4			6			4			1																																											
											1			5			10			10			5			1																																									
										1			6			15			20			15			6			1																																							
									1			7			21			35			35			21			7			1																																					
								1			8			28			56			70			56			28			8			1																																			
							1			9			36			84			126			126			84			36			9			1																																	
						1			10			45			120			210			252			210			120			45			10			1																															
					1			11			55			165			330			462			462			330			165			55			11			1																													
				1			12			66			220			495			792			924			792			495			220			66			12			1																											
			1			13			78			286			715			1287			1716			1716			1287			715			286			78			13			1																									
		1			14			91			364			1001			2002			3003			3432			3003			2002			1001			364			91			14			1																							
	1			15			105			455			1365			3003			5005			6435			6435			5005			3003			1365			455			105			15			1																					
1				16				120				560				1820				4368				8008				11440				12870				11440				8008				4368				1820				560				120				16				1			

Prime numbers

	2	3	5	7	11	13	17	19	23
29	31	37	41	43	47	53	59	61	67
71	73	79	83	89	97	101	103	107	109
113	127	131	137	139	149	151	157	163	167
173	179	181	191	193	197	199	211	223	227
229	233	239	241	251	257	263	269	271	277
281	283	293	307	311	313	317	331	337	347
349	353	359	367	373	379	383	389	397	401
409	419	421	431	433	439	443	449	457	461
463	467	479	487	491	499	503	509	521	523
541	547	557	563	569	571	577	587	593	599
601	607	613	617	619	631	641	643	647	653
659	661	673	677	683	691	701	709	719	727
733	739	743	751	757	761	769	773	787	797
809	811	821	823	827	829	839	853	857	859
863	877	881	883	887	907	911	919	929	937
941	947	953	967	971	977	983	991	997	

Graphs

K-th Shortest path

```
struct edge
{
    int s, e, c; //start, end, cost
    bool operator <(const edge& e)const
    {
        return c < e.c;
    }
};

const int SIZE = 100; //max nodes number
int N, start, end, K; //find the k-th shortest path from start to end
int dist[SIZE][SIZE]; //this can be adjList instead of adjMatrix
//Returns -1 if no k-th shortest path exist between start and end
int getKthShortestPath()
{
    multiset<edge> pq; //first is cost and second is node
    edge e = {-1, start, 0};
```

```

    pq.insert(e);
    vector<int> reached[N]; //reached[i][j] is the cost of the j-th
shortest path from start to i
    while(!pq.empty())
    {
        edge e = *pq.begin();
        pq.erase(pq.begin());
        if(reached[e.e].size() >= K)
            continue;
        reached[e.e].push_back(e.c);
        for(int i = 0 ; i < N ; i++)
        {
            if(dist[e.e][i] == -1)
                continue;
            edge ne = {e.e, i, e.c + dist[e.e][i]};
            pq.insert(ne);
        }
    }
    //no k-th path exist between start and end 23
    return reached[end].size() >= K? reached[end].back(): -1;
}
//MAIN
memset(dist, -1, sizeof dist);
//set N, start, end, K, dist

```

Floyd

```

void printPath(int path[][N], int v, int u)
{
    if (path[v][u] == v)
        return;
    printPath(path, v, path[v][u]);
    cout << path[v][u] << " ";
}

// Function to run Floyd-Warshall algorithm
void FloydWarshall(int adjMatrix[][N])
{
    // cost[] and parent[] stores shortest-path
    // (shortest-cost/shortest route) information
    int cost[N][N], path[N][N];

    // initialize cost[] and parent[]
    for (int v = 0; v < N; v++)
    {
        for (int u = 0; u < N; u++)
        {

```

KMP Palindrome

```
while (cin >> s)
{
    int s3[10001];
    s2 = s;
    s3[0] = 0;
    reverse(s2.begin(), s2.end());
    for(int i = 1, j = 0; i < sz(s); i++)
    {
        while(j > 0 && s2[j] != s2[i])
            j = s3[j - 1];
        if(s2[j] == s2[i])
            j++;
        s3[i] = j;
    }
    int a = 0;
    for(int i = 0; i < sz(s); i++)
    {
        while(a > 0 && s[i] != s2[a])
            a = s3[a - 1];
        if(s[i] == s2[a])
            a++;
        if(a == sz(s))
            a = s3[a - 1];
    }
}
```

```

        while(curL < 1)
            remove(curL++);

        ret[j.i] = getAns();
    }
    return ret;
} a, b;

    cin >> a >> b;
    //ans = xor grundy of all the starting positions(all the piles)
    ans ^= solve(a, b);
}
    cout << (ans ? "Y" : "N");
    return 0;
}

```

MEX

```

bool used[10001];

void MEX()
{
    int mex=0;
    for (int k = 0; k <= (int)se.size() + 1; k++)//se : vector
        used[k] = 0;
    for (int k = 0; k < (int)se.size(); k++)
    {
        if (se[k]<=(int)se.size())
            used[se[k]] =1;
    }
    while (used[mex])
        mex++;
}

```

Note

if we can take 1, 2 or 3 coins from the pile :

grundy(10) = mex({grundy(9), grundy(8),grundy(7)}) //mex of the grundy numbers of the states that i can go to from here

Grundy of losing states = 0, winning states = inf (base case)

Result = xor(grundy of all the starting states/ piles)

Game of Nim

If we have k piles then sum all bits and if all of them is divisible by (k+1) first player lose .

Losing positing xor=0.

If losing is last to move if all numbers is ≤ 1 xor=0 first player wins else normal wins

Misere nim (last one play wins): if all 1 if n even First wins else second
Else normal nim

Articles

Aho corasick applications

1- find all occurrences of a set of strings in a text

2- Finding the lexicographical smallest string of a given length that doesn't match any given strings

A set of strings and a length L is given. We have to find a string of length L , which does not contain any of the string, and derive the lexicographical smallest of such strings.

We can construct the automaton for the set of strings. Let's remember, that the vertices from which we can reach a leaf vertex are the states, at which we have a match with a string from the set. Since in this task we have to avoid matches, we are not allowed to enter such states. On the other hand we can enter all other vertices. Thus we delete all "bad" vertices from the machine, and in the remaining graph of the automaton we find the lexicographical smallest path of length L . This task can be solved in $O(L)$ for example by depth first search.

3- Finding the shortest string containing all given strings

Here we use the same ideas. For each vertex we store a mask that denotes the strings which match at this state. Then the problem can be reformulated as follows: initially being in the state ($v=\text{root}$, $\text{mask}=0$), we want to reach the state (v , $\text{mask}=2^n-1$), where n is the number of strings in the set. When we transition from one state to another using a letter, we update the mask accordingly. By running a breath first search we can find a path to the state (v , $\text{mask}=2^n-1$) with the smallest length.

4- Finding the lexicographical smallest string of length L containing k strings

As in the previous problem, we calculate for each vertex the number of matches that correspond to it (that is the number of marked vertices reachable using suffix links).

We reformulate the problem: the current state is determined by a triple of numbers (v, len, cnt), and we want to reach from the state (root, 0, 0) the state (v, L, k), where v can be any vertex. Thus we can find such a path using depth first search (and if the search looks at the edges in their natural order, then the found path will automatically be the lexicographical smallest).

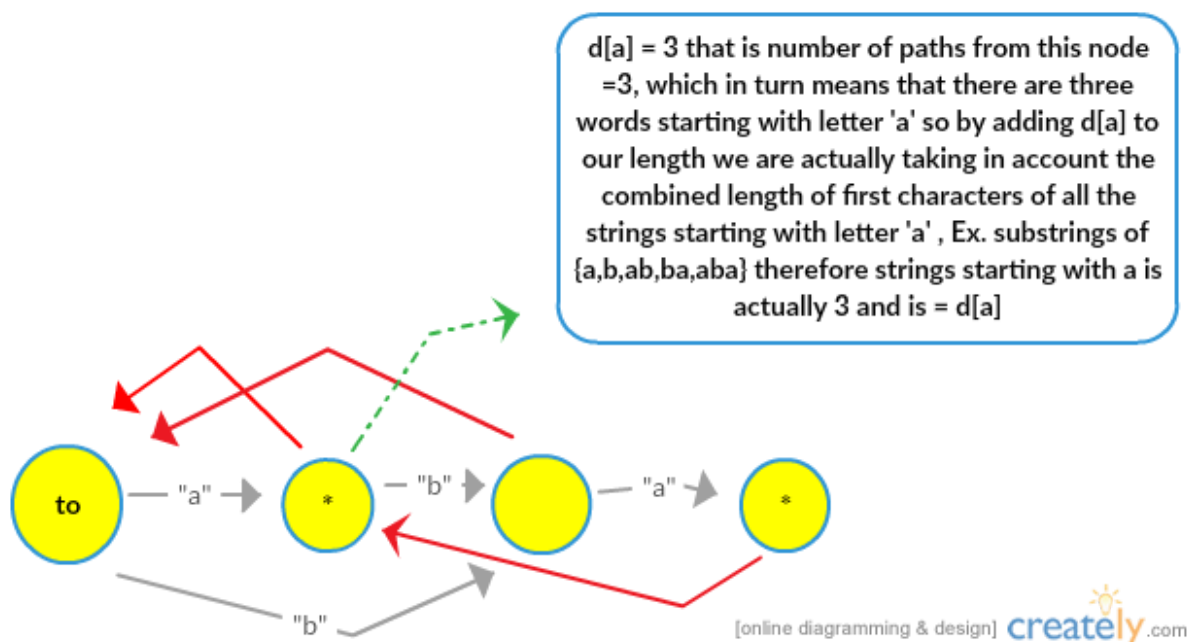
Suffix automaton applications

1. Finding number of distinct substrings (implemented above)

2. The Lexicographically kth substring (implemented above)

3. Total Length of all distinct substrings :

For this we need to look at things more closely. Consider



```
vector<int>ans(MAXLEN,0);
int lesub( int ver)
{
    int tp = 0; //To Find Words just add length of words instead of 1
    if(ans[ver])
        return ans[ver];
    for(int i = 0; i < 26; i++)
        if(st[ver].next[i])
            tp = lesub(st[ver].next[i]) + d[st[ver].next[i]];

    ans[ver] = tp;
    return ans[ver];
}
```


4. Smallest Cyclic Shift to obtain lexicographical smallest of All possible

Consider string “aba” then the minimum cyclic shift is ‘1’ because by shifting the pattern by one we get string “aab” which is lexicographically smallest of all possible (“aba” , “aab”, “baa”) . For solving this problem we build the suffix automaton of “String + String” Now the problem is reduced to above problem where $k = 1$, solving until we get substring of length = length of given string, Don’t worry about the fpos part that is done to find the start position of minimum lexicographical string so that we can find number of shifts.

```
int tp = 0 ;
string s;
int t = s.length();
s = s + s;
void minshift(int ver)
{
    for(int i = 0; i < 26; i++)
        if(st[ver].next[i])
        {
            tp++;
            if(tp == t)
            {
                cout<<st[ver].fpos - t + 2<<endl;
                Break;
            }
            minshift(st[ver].next[i]);
            break;
        }
}
```

5. POSITION OF FIRST OCCURRENCE

Given a text T and we receive request of the form : given a pattern P we need to know the position of first occurrence of P in T .

To solve this we will build the suffix automaton of the Text T , we also need to add in the preprocessing finding $fpos$ (first position of occurrence) for all states of automaton, such that $fpos[cur] = len[cur] - 1$ and $fpos[clone] = fpos[q]$, then answer to our query is simply equal to $fpos[t] - p.length() + 1$ where " t " is the state corresponding to pattern , p is the pattern. Here is the code

```
st[cur].fpos = st[cur].len - 1;
st[clone].fpos = st[q].fpos;
at appropriate positions in sa_extend function */

int firstpostn()
{
    int s_t = 0 ;
    int vtx = 0;
    int u = s_t;
    while(s_t < p.length())
    {
        vtx = st[vtx].next[p[s_t] - 'a'];
        s_t++;
    }
    Cout << st[vtx].fpos - p.length() + 1 << endl;
}
```

6. COUNT NUMBER OF OCCURRENCES

To solve this we need to build suffix automaton of S , along with this we also need to make a preprocessing for each state v to find $count[v]$ which is equal to the size of $endpos(v)$ since it would not be possible to store all suffixes in $endpos(v)$, we will instead store its size. For each state , other than initial state or states obtained by cloning we initially assign $count[v] = 1$. Then we will go over all the states in descending order of their length len and calculate $count[v]$ using $count[link[v]] += count[v]$. After this answer to query is $count[t]$ where " t is the state of the pattern " . Here is the code which find occurrences of pattern string " pa " in main string " s ".

```

set<pair<int, int> base;
int s_t = 0
/* ADD these 4 lines at appropriate places in sa_extend function
   cnt[cur] = 1;
   base.insert(make_pair(st[cur].len, cur));
   cnt[clone] = 0;
   base.insert(make_pair(st[clone].len, clone));
*/
int repstr()
{
    set::reverse_iterator it;
    for(it = base.rbegin(); it != base.rend(); it++)
        cnt[st[it->second].link] += cnt[it->second];
    int vtx = 0;
    while(s_t <= pa.length()-1)
    {
        vtx = st[vtx].next[pa[s_t] - 'a'];
        s_t++;
    }
    cout << cnt[vtx] << endl;
}

```

7. LONGEST COMMON SUBSTRING OF TWO STRINGS

We now go on line T and look for each prefix of the longest suffix prefix S occurring. In other words, for each position in the line T we want to find of the longest common substring S and T ending at that position. To do this, we will support the two variables: the current state v and the current l length. These two variables will describe the current matching part: its length and state that corresponds to it.

Initially, $p = t_0$ and $l = 0$ i.e. empty match .

Now let us consider the character $T[i]$ and we want to recalculate the answer for it.

- If the state of v the automaton has a transition with label $T[i]$, we simply commit this change and increase l by one.
- And if the state v is not of the desired transition, then we should try to shorten the current matching portion for which it is necessary to go to suffix link:

$$v = \text{link}(v).$$

$$l = \text{len}(v).$$

Unless we find a new state with required transition or we reach a fictitious

state -1 , we have to go on the suffix link. The answer to the problem will be a maximum of the values l for all time rounds.

```
string lcs (string s, string t)
{
    sa_init();
    for (int i=0; i<(int)s.length(); ++i)
        sa_extend (s[i]);
    int v = 0, l = 0, best = 0, bestpos = 0;
    for (int i = 0; i < (int)t.length(); ++i)
    {
        while (v && ! st[v].next.count(t[i]))
        {
            v = st[v].link;
            l = st[v].length;
        }
        if (st[v].next.count(t[i]))
        {
            v = st[v].next[t[i]];
            ++l;
        }
        if (l > best)
            best = l, bestpos = i;
    }
    return t.substr (bestpos-best+1, best);
}
```

Number of different substrings

We preprocess the string s by computing the suffix array and the LCP array. Using this information we can compute the number of different substrings in the string.

To do this, we will think about which new substrings begin at position $p[0]$, then at $p[1]$, etc. In fact we take the suffixes in sorted order and see what prefixes give new substrings. Thus we will not overlook any by accident.

Because the suffixes are sorted, it is clear that the current suffix $p[i]$ will give new substrings for all its prefixes, except for the prefixes that coincide with the suffix $p[i-1]$. Thus, all its prefixes except the first $\text{lcp}[i-1]$ one. Since the length of the current suffix is $n-p[i]$, $n-p[i]-\text{lcp}[i-1]$ new suffixes start at $p[i]$. Summing over all the suffixes, we get the final answer

```

        g[it][g[from[it]][fromEdge[it]].backEdge].flow -=
addFlow;

        cost += g[from[it]][fromEdge[it]].cost * addFlow;

        it = from[it];
    }

    flow += addFlow;
}

return {cost, flow};
}
};

```

Probability

Ncr Iterative (Without Mod)

```

for (int i=0;i<=100;i++)
{
    ncr[i][0]=ncr[i][i]=1;
    for (int k=1;k<i;k++)
    {
        ncr[i][k]=ncr[i-1][k]+ncr[i-1][k-1];
    }
}
// max 100

```

Notes and rules

In some cases, the first event happening impacts the probability of the second event. We call these **dependent events**.

In other cases, the first event happening does not impact the probability of the seconds. We call these **independent events**.

Conditional Probability

$$P(A \text{ and } B) = P(A) \cdot P(B | A)$$

If the events are independent, one happening doesn't impact the probability of the other, and in that case $P(B | A) = P(B)$

Expected Value

$$E(x) = p(x) \cdot x$$

$$E(x+y) = E(x) + E(y)$$

$$E(x \cdot y) = E(x) \cdot E(y)$$

$$E(x+c) = E(x) + c$$

$$E(x \cdot c) = E(x) \cdot c$$

For continuous random variable $E(x) = \int_{-\infty}^{\infty} x \cdot f(x) dx$ (from zero to x)

Conditional expected value

$$E[X | Y=y] = \sum_x P(X=x | Y=y) \cdot x$$

Linear Expectation

$$E[1_t] = P(t \text{ occurs}) \cdot 1 + P(t \text{ does not occur}) \cdot 0 = P(t \text{ occurs}).$$

Infinite Probability

$$p(x) = (1-p(x))^x$$